LEMMA: Learning from Errors for MatheMatical Advancement in LLMs

Zhuoshi Pan , Yu Li , Honglin Lin , Qizhi Pei , Zinan Tang , Wei Wu , Chenlin Ming ,[1]

H. Vicky Zhao1, Conghui He2,*, Lijun Wu2,*

Abstract. Large language models (LLMs) have demonstrated remarkable reasoning capability in solving mathematical problems. However, existing approaches primarily focus on improving the quality of correct training data, e.g., distilling high-quality correct solutions from advanced models, neglecting the value contained in error data, potentially hindering the model's reflective ability. Though some studies attempt to leverage error data, they often involve complex mechanisms, such as Monte Carlo Tree Search (MCTS) to explore error nodes. In this work, we propose to enhance LLMs' reasoning ability by Learning from Errors for MatheMatical Advancement (LEMMA). LEMMA constructs data consisting of an incorrect solution with an erroneous step and a reflection connection to a correct solution for fine-tuning. Specifically, we systematically analyze the model-generated error types and introduce an error-type grounded mistake augmentation method to collect diverse and representative errors. Correct solutions are either from fixing the errors or generating a fresh start. Through a model-aware smooth reflection connection, the erroneous solution is transferred to the correct one. By fine-tuning on the constructed dataset, the model is able to self-correct errors autonomously within the generation process without relying on external critique models. Experimental results demonstrate that LEMMA achieves significant performance improvements over other strong baselines.

[1]Corresponding Authors. [2]Work during internship at Shanghai AI Lab. [3]Code: ¡https://github.com/pzs19/LEMMA¿

1 Introduction

Recently, Large Language Models (LLMs) have significantly improved their ability to solve mathematical problems through Supervised Fine-Tuning (SFT). A common strategy involves refining the quality of chain-of-thought (CoT) reasoning data, such as distilling high-quality solutions from advanced models [30, 59] . While these methods enhance the model's capacity to generate step-by-step solutions, they predominantly focus on optimizing correct reasoning trajectories while overlooking the potential of error data. This omission limits the model's

ability to learn from mistakes, thereby constraining its reflective reasoning capability. Reflection—the process of identifying, analyzing, and correcting errors—is a critical component of human problem-solving [42] . Given the failure to integrate this ability into LLMs, models remain vulnerable to propagating errors during inference without autonomous correction mechanisms.

To address this gap, recent studies have begun exploring methods to cultivate reflection in LLMs by leveraging error data. For instance, some works [54, 23, 36, 7] employ external critical models to critique intermediate reasoning steps or use Monte Carlo Tree Search (MCTS) to navigate complex reasoning paths and prune error branches. Others [56, 9, 64, 57] propose self-correction frameworks that construct incorrect-correct data for fine-tuning, enabling the model to iteratively revise its outputs. However, these approaches suffer from significant limitations. MCTS-based methods introduce substantial computational overhead and complexity, while self-correction methods rely on naive and inefficient techniques to collect incorrect and correct reasoning trajectories.

In this work, we propose LEMMA to Learn from Errors for MatheMatical Advancement, a novel method to systematically enhance LLMs' reflective reasoning by constructing and learning from error-corrective trajectories. Our approach begins with a fine-grained categorization of error types in model-generated solutions, ranging from "question misinterpretation" to "calculation error". Building on this taxonomy, we design an error-type grounded error augmentation strategy that diversifies error data by (1) harvesting mistakes from the target model's own reasoning traces and (2) guiding advanced models to generate representative errors according to the analyzed error type distributions. For each erroneous solution, we then construct paired reflection data through two complementary mechanisms: Fix & Continue Trajectories, where the mistake is directly corrected within its original context, and Fresh & Restart Trajectories, where a new correct solution is generated from scratch. These trajectories are seamlessly connected via model-aware reflection links—annotations that explain the error's origin and justify the correction—resulting in coherent training examples.

Experiments across mathematical reasoning benchmarks (e.g., GSM8K, MATH) demonstrate LEMMA's effectiveness. Models fine-tuned with LEMMA outperform both standard SFT baselines and prior error-aware methods (up to 13.3% average accuracy improvement on LLaMA3-8B). LEMMA-trained models also achieve strong generalization ability through evaluation on out-of-distribution (OOD) benchmarks. Further analysis reveals LEMMA can consistently reduce the occurrence of representative error types. In contrast, while fine-tuning on the original training set (SFT) improves overall accuracy, it leads to an increase in certain error types. These results validate that structured learning from errors, guided by systematic analysis, is a powerful yet underutilized lever for advancing mathematical reasoning in LLMs.

2 Related Work

Self-Improvement in Math Reasoning Due to the scarcity of mathematical reasoning data with detailed, human-annotated reasoning steps [41, 29] , some studies [63, 60, 33, 40, 45] leverage the correct output of the model itself for fine-tuning. This strategy is also known as self-improvement or reject sampling fine-tuning.

Recently, some works [35, 53, 55, 54, 36, 32, 7] have begun using Monte Carlo Tree Search (MCTS), Process Reward Models (PRM) [25] or critique models to further enhance self-improvement. However, these methods only utilize the correct solutions, neglecting the generated errors. Since models only learn from correct solutions, they struggle to reflect on and self-correct errors they made, leading to error accumulation [64, 9, 56]

Data Augmentation in Math Reasoning Data augmentation is also a prevalent strategy to enhance model performance on mathematical tasks. [30] and [61] distill reasoning capabilities from LLMs into smaller LMs. Dart-Math [46] introduces a difficulty-aware answer augmentation strategy, where more solutions are generated for harder problems. To further boost model performance, several works [43, 12, 62, 26, 47, 66, 5, 27, 18, 29, 20, 17] synthesize more training data by creating new mathematical problems and solutions. For example, LLM2LLM [17] iteratively synthesizes more data based on the data points that the student model fails to answer. MetaMath [58] combines answer augmentation and question augmentation, as well as two backward reasoning methods [14, 50] , to further augment training data. Although these works have acknowledged the value of erroneous data, their focus is on sample-level augmentation through the synthesis of additional data points. In contrast, our method implements sequence-level augmentation by enriching each data point with common errors and subsequent reflections, thereby fostering a more profound capacity for self-reflection and correction. Furthermore, our method is orthogonal to these question augmentation methods and can be directly integrated with them.

Reflection and Self-Correction in LLMs Reflection and self-correction mechanisms have been proven to be effective in enhancing the performance of large language models (LLMs) across various domains. To encourage models to identify and amend their previous errors, one common approach leverages feedback from an external verifier or critic model [2, 21, 44, 39, 38, 19, 6] . Alternatively, some research [50, 57, 64, 9, 56, 51, 52, 36, 65, 28, 16] focuses on fostering the self-correction capabilities of LLMs during the generation process itself, without external feedback. To gather training data for self-correction, prior studies introduce erroneous reasoning trajectory by employing a relatively high temperature [54, 56, 64, 9] or by sampling from multiple models [2] . However, recent research [28, 37] has shown that elevating the temperature can result in nonsensical errors or incoherent text that are unlikely to arise during typical generation scenarios. Additionally, sampling from multiple models [2] can introduce irrelevant errors

that the student model would not make. Moreover, these approaches neglect different correction strategies, which could potentially restrict the model's ability to reflect and self-correction.

3 Methodology

To better leverage generated reasoning errors for enhancing the self-reflection and correction capabilities of LLMs, we begin by conducting a systematic analysis of common error types in widely-used models. Building on this analysis, we introduce LEMMA, a novel approach that strategically constructs self-correction data to improve the mathematical reasoning abilities of LLMs. Fig. 2 provides an overview of the LEMMA framework.

3.1 Task Formulation

We begin by defining key components of LEMMA. The generated reasoning trajectory τ is a sequence of reasoning steps: $\tau = (s_1, s_2, \ldots, s_n, \hat{a})$, where $\hat{a}$ is the predicted answer. A bad trajectory includes both correct steps s^g, incorrect steps s^g and ends with an incorrect answer: $\tau^b = (s_1^g, \ldots, s_k^g, s_1^b, \ldots, s_m^b, \hat{a}^b)$. Models equipped with reflection and self-correction capabilities should be able to identify and rectify the incorrect steps $\{s_1^b, ..., s_m^b\}$, leading to a revised trajectory. The revised trajectory can be viewed as the concatenation of a bad trajectory, a Reflection Phrase (RP), and a correct trajectory τ^g, expressed as $\tau^r = \tau^b \oplus RP \oplus \tau^g$. Here, RP represents the reflection phrases that pinpoint and correct previous errors while seamlessly transitioning to a correct step. To minimize error accumulation, the model should recognize and correct errors as early as possible. Therefore, the bad trajectory τ^b should be a subsequence that ends at the first erroneous step, denoted as $\tau_{sub}^b = (s_1^g, \ldots, s_k^g, s_1^b)$. The following paragraphs will detail how we collect the bad sub-trajectory τ_{sub}^b and the good trajectory τ^g, ultimately constructing the error-corrective revision trajectory $\tau^r = \tau_{sub}^b \oplus RP \oplus \tau^g$ for model training.

3.2 Error Analysis

To gain a holistic understanding of the mathematical reasoning errors in common LLMs, we conduct a systematic analysis on error types. We use an error taxonomy modified from [22] , as detailed in Tab. 1 . Fig. **??** presents the distribution of error types for different models. Our key findings are: (1) The most common errors include "Question Misinterpretation (QM)", "Formula Confusion Error (FC)" and "Calculation Error (CA)". This indicates that the models require improvements in areas such as problem comprehension, formula application, and conceptual understanding. (2) The distribution of error types is relatively consistent across different models. These key insights serve as the foundation for our subsequent error-type grounded error augmentation method.

The above analysis is conducted using greedy decoding generation.[The temerature of softmax function is set to 0.] However, prior research [54, 56, 28, 64] typically uses a relatively high sampling temperature (e.g., $t = 1.0$ in [56] and $t = 1.1$ in [28]) to collect a diverse set of bad trajectories $\{\tau^b\}$. Hence, we also investigate the effect of temperature on error types. Fig. **??** (a) depicts how the distribution of error types varies with different softmax function temperatures. As the temperature increases, nonsensical errors, exemplified in Fig. **??** (b), begin to emerge. In other words, the occurrence of nonsensical errors rises with sampling temperature, whereas this type of error is generally absent in greedy decoding.

3.3 Erroneous Trajectory Collection

Based on our analysis of the relationship between error types and sampling temperature in Sec.

, we opt not to increase the sampling temperature. Instead, we employ a relatively low sampling temperature $t = 0.7$, which is widely used in mathematical evaluation [54, 56, 65] . To mitigate the reduced diversity of error steps at lower temperatures, we propose an error-type grounded mistake augmentation method that systematically generates diverse and meaningful errors for subsequent correction. Specifically, we first determine the error type distribution for each question based on our prior analysis. We then leverage a teacher model (GPT-4o) to intentionally produce erroneous trajectories given an error type, which is sampled from the previously obtained error type distribution for each question. This approach ensures that the introduced errors are both diverse and closely aligned with the errors observed using the student model.

However, as our objective is not to induce the model to learn flawed reasoning but rather to learn reflection, we also instruct the teacher model to annotate the first erroneous step within each incorrect trajectory τ^b. Then, the incorrect trajectory is truncated at this identified step to create a sub-trajectory $\tau^b_{sub} = (s^g_1, \ldots, s^g_k, s^b_1)$. This design guides the fine-tuned models to timely recognize and rectify their errors, preventing them from proceeding with reasoning from an erroneous step. Using this approach, we compile a comprehensive collection of bad trajectories, which consists of: (1) the erroneous trajectories generated by error augmentation and (2) those produced by the student model itself. This strategy mirrors the human learning process, where students not only reflect on and correct their own mistakes but also receive guidance from teachers, who highlight common error-prone steps based on the overall performance of all students. This error-type grounded mistake augmentation method sets our approach apart from prior work. It avoids the inclusion of meaningless or irrelevant errors generated by using a high sampling temperature [54, 56, 28, 64, 9] or collected from other models [2]

3.4 Revision Trajectory Generation

Upon obtaining the bad trajectory τ^b_{sub}, we proceed with a correction process to generate the final revision trajectory $\tau^r = \tau^b_{sub} \oplus RP \oplus \tau^g$. Inspired by the self-correction process in humans [11] , we explore two correction strategies:

(1) Fix & Continue Revision: In this strategy, the teacher model fixes the student model's first error step and continues the reasoning process to reach the correct answer. However, as illustrated in Fig. 9 , there are instances where, despite the initial reasoning steps being correct, they may not be a "smart" way to solve the problem. This can result in a prolonged reasoning trajectory involving complex reasoning and intensive computations, which are more susceptible to errors. To address this limitation, we introduce the "Fresh & Restart" correction strategy as follows.

(2) Fresh & Restart Revision: In this strategy, the teacher model critiques the student model's errors and then initiates the reasoning process anew, rather than continuing from an erroneous "intermediate" step. We encourage the model to explore alternative solutions using the prompt depicted in Fig. 15 in Appendix . This approach emulates human correction processes, where, upon realizing an initial approach is flawed, one may abandon the original reasoning steps and start anew instead of making minor adjustments to the first attempt.

By combining both correction strategies, we generate a diverse set of revision trajectories $\{\tau^r\}_{i=1}^N$. Training on the constructed data enables the student model to learn different self-correction strategies. Following [54] and [36] , we also employ the teacher model to smooth the entire revision trajectory, adding necessary logical transitions and connections to produce the final training data. Finally, we filter the trajectories based on the correctness of the final answer, retaining only those that lead to a correct answer.

4 Experiments

We evaluate our method through comprehensive experiments from three key aspects: (1) In-distribution mathematical tasks, (2) Out-of-distribution mathematical tasks, and (3) Reflective mathematical reasoning tasks.

4.1 Implementation Details

Trajectory synthesis. We use the training set of MATH [10] and GSM8K [3] to generate the error-corrective reasoning trajectory. We utilize LLaMA3-8B to produce the self-generated errors at a temperature of 0.7 and employ GPT-4o [13] as the teacher model to deliberately introduce errors and perform subsequent corrections. Additionally, we employ an open-source model, LLaMA-3.1-Nemotron-70B [48] , as an alternative teacher model to demonstrate the generalization of our method. For LEMMA (w/ MetaMath), we collect additional error-corrective reasoning trajectories based on the new questions of MetaMath [58] . For each error, we apply both "Fix & Continue" and "Fresh & Restart" correction strategies once. After filtering out the trajectories with incorrect final answers, we

obtain $88.90k$ error-corrective reasoning trajectories as training data. We fine-tune various base models, including general-purpose models such as LLaMA3-8B and Mistral-7B-v0.1, as well as the math-specialized model DeepSeekMath-7B and Qwen2-Math-7B. Further implementation details are available in Appendix

Benchmarks. We use GSM8K [3] and MATH [10] as the In-Domain evaluation. For Out-of-Domain evaluation, we choose ASDIV [31] , MAWPS [15] , Mathematics [4] , SVAMP [34] and College-Math [43] . Following [65] , we also adopt the Follow-up QA (FQA) and Error Correction (EC) tasks of MathChat [24] , which require the model to reflect on previous generation and perform further reasoning. Unless specified otherwise, we use Pass@1 as the evaluation metric. The performance results using majority voting are detailed in Appendix

Baselines. We compare LEMMA with four self-correction methods and four data augmentation approaches. For self-correction methods, we consider: (1) Intrinsic Self-Correction (ISC) [9] : Teaching small language models to self-correct by training on the constructed self-correction data. (2) S^3C-MATH [56] : Employing a step-level sampling approach to generate potentially erroneous steps, followed by reflection and improvement, to construct self-correction data. (3) RefAug [65] : Appending a "reflection" part to the original solution, which involves proposing an alternative solution and solving a similar problem. (4) RefAug-90k [65] : To eliminate the influence of sample size and the annotation model, we use the official code[¡https://github.com/ytyz1307zzh/RefAug¿] of RefAug to generate $89.82k$ data with GPT-4o, which aligns with LEMMA in terms of both sample size and annotation model.

For data augmentation approaches, we consider: (1) SFT: Training on the union of GSM8K and MATH training set. (2) Rejection Sampling Fine-tuning (RFT) [60] : Training on the correct self-generated reasoning trajectories. (3) MetaMath [58] : Combining answer augmentation, question rephrasing, and two backward reasoning methods [14, 50] , to augment training data. (4) GPTAug: Prompting GPT-4o to generate step-by-step solution for each question. Appendix

discusses more details regarding baseline implementation.

4.2 Main Result

Tab. 2 lists the performance of different methods. We summarize the key findings as follows.

(1) LEMMA significantly outperforms baseline methods across most tasks, achieving an average accuracy improvement of at least 2.3% for LLaMA3 and 4.5% for DeepSeekMath. The enhancement is particularly noticeable in challenging tasks such as MATH [10] and College-Math [43] , where LEMMA surpasses baselines by at least 4.1% and 2.8% on LLaMA3, respectively. This underscores the efficacy of reflective and self-correction capabilities for solving complex math problems. Note that although Dart-Math [46] achieves superior performance, our method is not directly comparable to DART-Math due to the differences in the

teacher model and data size. In addition, while LEMMA lags slightly behind DART-Math on conventional math tasks, it performs better on tasks requiring reflection and follow-up reasoning, such as the follow-up QA (FQA) and Error correction (EC) tasks of MathChat [24] as shown in Tab. 3 . (2) Interestingly, RFT [60] lags behind all reflection and self-correction methods. We attribute this to the inherent limitation of RFT, which solely utilizes the correct self-generated solutions, forgoing the valuable opportunity to learn from failures. (3) Additionally, LEMMA demonstrates strong performance across both in-distribution and out-of-distribution datasets. While some baselines, such as MetaMath, achieve relatively good results on in-distribution datasets, they fall short compared to LEMMA on out-of-distribution datasets. Notably, scaling the data size of RefAug [65] to $89.82k$ data (i.e., RefAug-90k) enhances in-distribution performance; however, the improvements on out-of-distribution datasets are limited or even negative.

4.3 Reflective Math Reasoning Performance

Following [65] , we assess the reflective reasoning abilities of LLMs fine-tuned via various methods. Tab. 3 presents the results. Notably, LEMMA significantly enhances the reflective reasoning capabilities of models compared to other data augmentation methods, achieving improvements of at least 3.3% and 4.1% in accuracy on MathChat-FQA-3rd and MathChat-EC, respectively. Although some data augmentation approaches, such as MetaMath, have achieved considerable performance gains on multi-turn math question answering (i.e., MathChat-FQA), they fall short in improving error correction ability, with only a 0.5% accuracy increase on MathChat-EC compared to SFT. In comparison to reflection and self-correction methods, such as ISC and RefAug, LEMMA also demonstrates notable superiority. For instance, LEMMA surpasses ISC by 4.1% and 6.2% accuracy points on MathChat-FQA-3rd and MathChat-EC, respectively. Fig. 10 and Fig. 11 in Appendix

show some output cases where the model fine-tuned with LEMMA performs reflection and self-correction to produce more accurate answers. These results further underscore LEMMA's advantages in advancing reflective and self-correction capabilities of LLMs.

4.4 Integration with DART-Math

LEMMA is not contradictory but rather complementary to DART-MATH. Inspired by their work, we synthesize more error-corrective solutions for more challenging problems. Specifically, we calculate the failure rate of the LLaMA3-8B model to determine the number of solutions to synthesize for each sample. For each problem, we generate $n = f \times k_{max}$ data points, where f is the failure rate and k_{max} is the maximum number of solutions per question. Among these error-corrective solutions, half of the errors are generated by the student model itself, and the other half are introduced by the teacher model. We set k_{max} to

20, resulting in a total of $90.7k$ samples. We refer to this setting as LEMMA-Hard. The results presented in Tab. 4 demonstrate that this setting improves the performance of our LEMMA, outperforming the model trained on a $90.7k$ dataset downsampled from DART-Math-Hard by an average accuracy gain of 1.8%, and is comparable to models trained on the complete $585k$ DART-Math-Hard dataset.

4.5 Choice of Teacher Model

We also evaluate the performance of our approach using an open-source teacher model, LLaMA-3.1-Nemotron-70B, instead of GPT-4o, as the teacher model. The results, as shown in Tab. 8 of Appendix

, demonstrate that LEMMA continues to hold a significant advantage over baseline methods even after the replacement of the teacher model. This suggests that the improvements offered by LEMMA are not attributable to the teacher model itself, but rather to the efficacy of the systematic error introduction and correction strategy.

5 Analysis

5.1 Analysis on the Effect of Sample Size

We examine the impact of sample size on the performance of different methods. The results presented in Fig. **??** , highlight several key observations. (1) LEMMA consistently achieves superior performance across various sample sizes. Notably, as the dataset size increases, the performance gap between LEMMA and other baselines widens, underscoring its scalability potential. (2) LEMMA demonstrates stable performance improvements on both in-distribution (MATH) and out-of-distribution (Mathematics) datasets as the data size grows. In contrast, some baseline methods, such as ISC and RefAug, although showing gains on in-distribution datasets like MATH, tend to plateau or even decline in performance on out-of-distribution datasets. This saturation suggests that these methods might overfit to in-distribution data, lacking the generalization capabilities that LEMMA provides.

5.2 Analysis on Error Type after Fine-tuning.

We analyze the types of errors generated by the model before and after fine-tuning with LEMMA. We report the error count of the model before fine-tuning (Base), the model fine-tuned on the original dataset (SFT), and the model fine-tuned with our LEMMA approach. The results presented in Fig. **??** reveal several insightful trends. Firstly, LEMMA consistently reduces the occurrence of common error types, particularly in categories such as "Question Misinterpretation (QM)" and "Calculation Error (CA)". Secondly, although fine-tuning with the original training data (SFT) improves overall accuracy, it leads to an increase in certain error types, such as "Confusing Formula Error (FC)". This can

be attributed to limitations in the original training data, which may fail to address specific error patterns and potentially cause overfitting to certain reasoning paths.

5.3 Ablation study

We perform ablation studies to evaluate the contribution of the two core innovations of LEMMA: (1) the error-type grounded mistake augmentation method (Sec.
) and (2) the mixed correction strategies (Sec.
). To evaluate the effectiveness of the mistake augmentation module, we exclude it and rely solely on errors generated by the model itself, which we denote as "w/o Error Aug". To ensure that any performance degradation is not simply attributable to a smaller sample size, we generate the same number of revision trajectories as LEMMA, which we refer to as "w/o Error Aug (90k)". We also perform ablation on the error correction strategy by removing the "Fresh & Restart" method from the revision process, labeled as "w/o Fresh & Restart" and "w/o Fresh & Restart (90k)". Lastly, "w/o Truncation" means that we do not truncate the flawed trajectory at the first identified erroneous step, which aims to verify the necessity of the early truncation of the erroneous reasoning trajectory. We report the accuracy on in-distribution tasks in Tab. 5 ; for out-of-distribution performance, please refer to Tab. 11 in Appendix
. It is clear that removing the "error augmentation module" results in a significant performance drop. This decline is not due to sample size, as "w/o Error Aug (90k)" also exhibits a 6.2% accuracy decrease in performance on MATH compared to LEMMA. We attribute this decline to the reduced diversity of error steps, as the model relies solely on errors generated by the student model itself. In contrast, the error augmentation module introduces a variety of meaningful errors, enhancing the model's ability for reflection and self-correction. Furthermore, excluding the "Fresh & Restart" strategy degrades performance. This decline highlights the essential role of the "Fresh & Restart" correction: by enabling the model to reset and reassess problem-solving pathways, it significantly enhances mathematical reasoning capabilities.

6 Conclusion

In this work, we introduce LEMMA, a novel framework designed to enhance the mathematical reasoning capabilities of LLMs by systematically learning from errors. Based on a comprehensive analysis of error types, LEMMA employs an error-type grounded mistake augmentation strategy and constructs diverse revision pathways using both the Fix & Continue and Fresh & Restart correction strategies. This framework allows models to autonomously detect and correct errors during the generation process, thereby improving their mathematical reasoning abilities. Extensive experiments demonstrate that LEMMA significantly outperforms strong baselines.

7 Limitations

While LEMMA represents an advancement in enhancing the mathematical reasoning capabilities of LLMs, several limitations persist. Firstly, the synthesized dataset used in LEMMA comprises fewer than $90k$ examples, which is relatively small compared to data augmentation methods like MetaMath. This raises questions about whether an increase in data size could further enhance the performance. Moreover, as LEMMA fine-tuned models have exhibited preliminary reflection capabilities—akin to the "aha moment" of DeepSeek-R1 [8] , they may be good starting points for rule-based RL training. Exploring the use of LEMMA with rule-based RL could be a future direction.

8 Acknowledgments

This research was supported by National Key R&D Program of China (2022ZD0160201).

9 Appendix

9.1 Additional Experiment

Experiment on Other Base Models To further validate the robustness of our method across different models, we conduct additional experiments using Mistral-7B-v0.1 and Qwen2-Math-7B as base models. The results, presented in Tab. 9 , demonstrate that LEMMA consistently outperforms baseline methods on these models. Specifically, LEMMA achieves an average accuracy improvement of at least 2.9% on Mistral-7B-v0.1 and 3.5% on Qwen2-Math-7B. These consistent performance gains across different base models reinforce the robustness of our approach, highlighting LEMMA's efficacy in enhancing mathematical reasoning capabilities across a diverse range of models.

Experiment using Other Teacher Model To facilitate the community, we evaluate the performance of our approach using an open-source teacher model, LLaMA-3.1-Nemotron-70B, instead of GPT-4o. The results, presented in Tab. 8 , indicate that although there is a performance decrease when replacing the teacher model, LEMMA still maintains a significant advantage over baseline methods. This indicates that the improvements achieved by LEMMA do not stem from the teacher model itself but are primarily due to the effectiveness of the systematic error introduction and correction strategy. The consistent improvement further underscores the robustness of LEMMA.

Evaluation using Majority Voting We present the accuracy results under the Majority@32 setting in Tab. 10 . The results demonstrate that LEMMA consistently outperforms baseline methods in both Pass@1 and Majority@32

settings across most tasks. Notably, the Majority@32 setting significantly enhances LEMMA's accuracy compared to Pass@1, particularly on more challenging datasets such as MATH. For instance, LEMMA achieves improvements of 14.8% and 13.0% on MATH for LLaMA3 and DeepSeek-Math, respectively. In contrast, some baseline methods exhibit limited gains under the Majority@32 setting. For example, RefAug-90k shows only 7.6% and 8.1% improvements on MATH for LLaMA3 and DeepSeek-Math, respectively. These findings further underscore LEMMA's superiority and its compatibility with majority voting.

Ablation Study on Out-of-Distribution Datasets We report the ablation performance on both in-distribution and out-of-distribution tasks in Tab. 11 . The results show that removing either the "error augmentation" module or the "Fresh & Restart" correction strategy degrades performance, validating the design of our LEMMA approach.

Error Type Analysis on GSM8K In this section, we examine the error types on the GSM8K dataset. We observe a similar trend to that on MATH: the distribution of error types is consistent across different models. However, unlike MATH, the primary error types on GSM8K are "Question Misinterpretation (QM)", "Calculation Error (CA)" and "Confusing Concept Error (CC)", while "Formula Confusion Error (FC)", which is common on MATH, is less frequent on GSM8K. This difference stems from the inherent distinctions between the GSM8K and MATH datasets. The MATH dataset is more challenging, often involving complex mathematical formulas, whereas GSM8K is relatively simpler, with many problems requiring only basic arithmetic operations rather than the application of formulas. As a result, formula-related errors are less common on GSM8K.

In Fig. **??** , we present the changes in error types on GSM8K before and after fine-tuning. The results align with those observed on MATH, demonstrating that LEMMA consistently reduces the frequency of all error types. In contrast, while the overall performance of the model improves after fine-tuning with the original training data (SFT), certain specific type of error increase. This further highlights LEMMA's ability to systematically address and mitigate a wide range of errors, leading to more robust and reliable mathematical reasoning capabilities.

9.2 Case Study

Reflection and Self-Correction Output In Fig. 10 and Fig. 11 , we present examples from GSM8K and MATH, respectively, showcasing the outputs of the LLaMA3 model fine-tuned with our LEMMA data. These examples demonstrate that LEMMA model consciously identify potential errors in its previously generated steps, reflecting upon them and making necessary corrections, or verifying its answers before reaching a final conclusion. This ability explains why our method significantly improves accuracy in mathematical tasks: by enhancing the model's reflection and correction skills, it can ultimately rectify mistakes

and arrive at the correct answer, even if it initially takes a wrong approach or makes careless errors along reasoning path.

Full Error Taxonomy We present the full error taxonomy in Table 12 . Building upon the taxonomy proposed by [22] , we introduce additional error categories to enable a more granular identification of error types. Specifically, we add "Question Misinterpretation Error (QM)", "Confusing Concept Error (CC)", and "Nonsensical Output (NO)" to better capture the diverse range of errors that can occur during mathematical reasoning. The expanded taxonomy provides a structured framework for systematically categorizing and addressing the various types of errors encountered in mathematical problem-solving.

Error Type and Corresponding Examples In this section, we present examples of different error types generated by the model. As shown in Fig. 12 , we display the problem, the model-generated incorrect answer, the error type label assigned by the model, as well as the model's explanation for the label of representative error types. It can be observed that the model accurately identifies the first error type. Each error type exhibits distinct characteristics, clearly differentiating them from one another.

Smart Solution v.s. Brute Force Solution In Fig. 9 , we illustrate two typical solutions for solving a given problem: a smart solution and a brute force solution. While the brute force method starts with accurate initial steps, it requires complex calculations in the following steps. If the model initially fails to identify the smart solution, simply correcting the first incorrect step in the brute force solution does not easily lead to the correct final answer due to the complexity of subsequent calculations. Consequently, we propose the "Fresh & Restart" correction strategy, which encourages the teacher model to reconsider and generate new solutions. This strategy enables the model to learn a variety of correction techniques, thereby allowing it to rectify errors more flexibly.

9.3 Experiment Setup

Implementation Details of LEMMA We construct the incorrect-correct reasoning trajectories on the training set of MATH [10] and GSM8K [3] . We use GPT-4o [13] as the teacher model in our main experiment. Additionally, we employ an open-source model, LLaMA-3.1-Nemotron-70B [48] , as an alternative teacher model to demonstrate the generalization of our method, which produces similar results, as shown in Tab. 8 . To collect incorrect-correct reasoning trajectories based on the questions in the MetaMath dataset, we use LLaMA-3.1-Nemotron-70B as the teacher model to reduce computational costs, given that MetaMath is significantly larger than MATH [10] and GSM8K [3] . For trajectory synthesis, we use nucleus sampling with a temperature of 0.7 and top_p of 1.0. Based on our synthesized data, we fine-tune a wide range of base models, including general-purpose models such as LLaMA3-8B and Mistral-7B-v0.1, as well

as the math-specialized model DeepSeekMath-7B and Qwen2-Math-7B. We use the LLAMA-Factory package [¡https://github.com/hiyouga/LLaMA-Factory¿] for model training. We adopt a learning rate of 1e-5 with a warmup ratio of 0.03. We employ a cosine learning rate scheduler and set the gradient accumulation step to 8 to ensure stable training. All models are trained for 3 epochs. For evaluation, we use official evaluation package in Qwen2.5-Math repository [¡https://github.com/QwenLM/Qwen2.5-Math/tree/main/evaluation¿]. We set the maximum number of generated tokens to 1024 and the temperature to 0 for the Pass@1 metric. For the majority voting setting, we set the temperature to 0.7 and top_p to 1.0.

All our experiments were conducted on a server equipped with 8 x A100 GPUs. Training LLaMA3-8B on our synthesized dataset takes approximately 5 hours.

Implementation Details of Baselines We compare LEMMA with four self-correction methods and four data augmentation approaches. For self-correction methods, we consider: (1) Intrinsic Self-Correction (ISC) [9] : Teaching small language models to self-correct by training on the constructed self-correction data. In our reimplementation, we employ GPT-4o instead of GPT-3.5-Turbo to construct the self-correction data, ensuring the improvements are not attributed to model discrepancy. We synthesize $86.78k$ data in total, which aligns with LEMMA in quantity, to guarantee a fair comparison. Because the original prompt from their paper, "Please select the correct option from the provided choices and offer a comprehensive problem-solving process" is designed for multi-choice problems. We adapt this to our needs by using the prompt, "Below is an instruction that describes a task. Write a response that appropriately completes the request. ### Instruction: instruction ### Response: Let's think step by step" for the initial chain-of-thought (COT) generation. We then follow their official prompt "the answer of [Question] is [Ground-Truth]. Please provide a step-by-step explanation for resolving the given problem" to generate the correct solution using GPT-4o. (2) S^3C-MATH [56] : Employing a step-level sampling approach to generate potentially erroneous steps, followed by reflection and improvement, to construct self-correction data. Note S^3C-MATH synthesizes a total of $927k$ data based on MetaMath training set. Therefore, it should be compared with LEMMA (w/ MetaMath). (3) RefAug [65] : Appending a "reflection" part to the original solution, which involves proposing an alternative solution and solving a similar problem. We use the officially released "reflection" data and augment it with approximately the same amount of synthetic solutions generated by GPT-4-Turbo, as this configuration yields the best results in their paper. Our reimplementation of RefAug on most tasks is slightly better than the original results reported in their paper. (4) RefAug-90k [65] : To eliminate the influence of sample size and the annotation model, we employ the official code[¡https://github.com/ytyz1307zzh/RefAug¿] of RefAug to generate three correct reflection sections and three correct solutions for each

question-answer pair using GPT-4o. This produces 89.82k data, which aligns with our approach in terms of both sample size and annotation model.

For data augmentation approaches, we consider: (1) SFT: Training on the union of GSM8K and MATH training set. (2) Rejection Sampling Fine-tuning (RFT) [60] : Training on the correct self-generated reasoning trajectories. We collect a total 86.52k of data, which aligns with our LEMMA in quantity, isolating the impact of sample size. (3) MetaMath [58] : Combining answer augmentation, question rephrasing, and two backward reasoning methods, FOBAR [14] and Self-Verification [49] , to augment training data. (4) GPTAug: Prompting GPT-4o to generate step-by-step solution for each question. We generate a total 88.92k of data, consistent in quantity with our LEMMA, to ensure fair comparison.

Please refer to Tab. 7 for an overview of data statistics of the different methods.

9.4 Prompt

In Fig. 13 , Fig. 14 , and Fig. 15 , we present the prompts used for error injection, Fix & Continue correction, and Fresh & Restart correction, respectively. The prompts are designed to guide the teacher model in generating erroneous trajectories and correcting them using the two distinct strategies outlined in our methodology.

9.5 Comparison with Additional Baselines

To further illustrate the advantages of our method, we conduct additional experiments comparing LEMMA with additional baselines. The results are in Tab. 6 . Among these baselines, [2] bears the most similarity to ours, as it synthesizes mistake-correction data pairs by instructing GPT-4 to rectify incorrect reasoning paths generated by the reasoning model. However, it relies on ensembling errors from multiple LLMs to create diverse inaccurate reasoning paths, which can be prone to generating irrelevant inaccurate reasoning paths that the student model would not typically make. Furthermore, their method neglects different correction strategies, which restricts the model's ability to reflect and self-correction. Our two core innovations—(1) the error-type grounded mistake augmentation method and (2) mixed correction strategies ("Fix & Continue" and "Fresh & Restart")—represent novel contributions that have not been explored in previous work. To reimplement [2] , we use the same prompt as them, but change the student model / teacher model from LLaMA2 / GPT-4 to LLaMA3 / GPT-4o to align with our setting. We generate correction data based on the training set of GSM8K and MATH. [44] constructs a dataset called "CoTErrorSet" includeing errors and PaLM2-annotated error causes. They also incorporate fixed prefixes ([CORRECT RATIONALE] and [INCORRECT RATIONALE]) in fine-tuning, which they refer to as "Mistake Tuning". However, their method does not concatenate the incorrect rationale and correct rationale, but instead adds a specific prefix in front of the rationale, which is different from ours. For Mistake-tuning

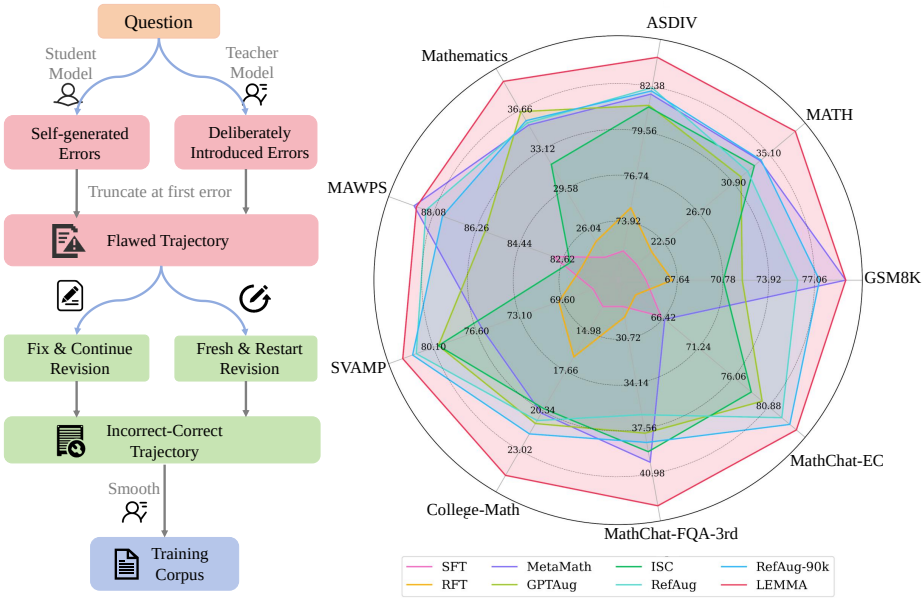


Fig. 1. Overview of LEMMA.

[44] , since the "CoTErrorSet" in Mistake-tuning does not include any data from MATH and is generated by PaLM2, we regenerate it based on the training set of MATH and GAM8K using GPT-4o (the same as ours) and then perform the mistake tuning. LLM2LLM [17] iteratively synthesizes more data based on the data points that the student model fails to answer. Their focus is on sample-level augmentation through the synthesis of additional data points. In contrast, our method implements sequence-level augmentation by enriching each data point with common errors and subsequent reflection. We also reimplement LLM2LLM [17] using GPT-4o as the teacher model to ensure a fair comparison. The results in Tab. 6 further illustrate the advantages of our method over these baselines.

References

1. Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv preprint arXiv:2303.08774.
2. Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. 2023. Learning from mistakes makes llm better reasoner. arXiv preprint arXiv:2310.20689.
3. Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168.

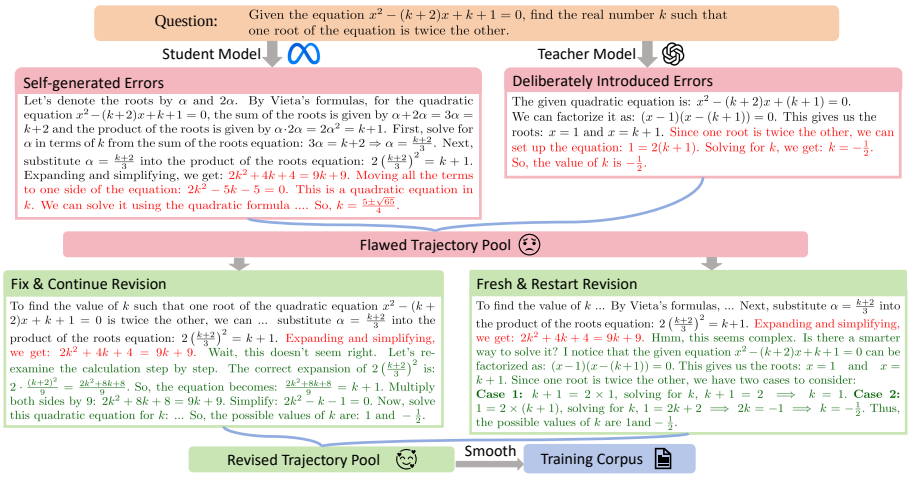


Fig. 2. The LEMMA framework. LEMMA uses an error-type grounded mistake augmentation module, and explores two error correction strategies to construct the incorrect-correct revision trajectory as training corpus.

4. Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, et al. 2021. Advancing mathematics by guiding human intuition with ai. Nature, 600(7887):70–74.
5. Yuyang Ding, Xinyu Shi, Xiaobo Liang, Juntao Li, Qiaoming Zhu, and Min Zhang. 2024. Unleashing reasoning capability of llms via scalable question synthesis from scratch. arXiv preprint arXiv:2410.18693.
6. Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In Forty-first International Conference on Machine Learning.
7. Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. 2025. rstar-math: Small llms can master math reasoning with self-evolved deep thinking. arXiv preprint arXiv:2501.04519.
8. Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948.
9. Haixia Han, Jiaqing Liang, Jie Shi, Qianyu He, and Yanghua Xiao. 2024. Small language model can self-correct. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 18162–18170.
10. Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. NeurIPS.
11. Michael HG Hoffmann. 2018. Stimulating reflection and self-correcting reasoning through argument mapping: Three approaches. Topoi, 37:185–199.
12. Yiming Huang, Xiao Liu, Yeyun Gong, Zhibin Gou, Yelong Shen, Nan Duan, and Weizhu Chen. 2024. Key-point-driven data synthesis with its enhancement on mathematical reasoning. arXiv preprint arXiv:2403.02333.

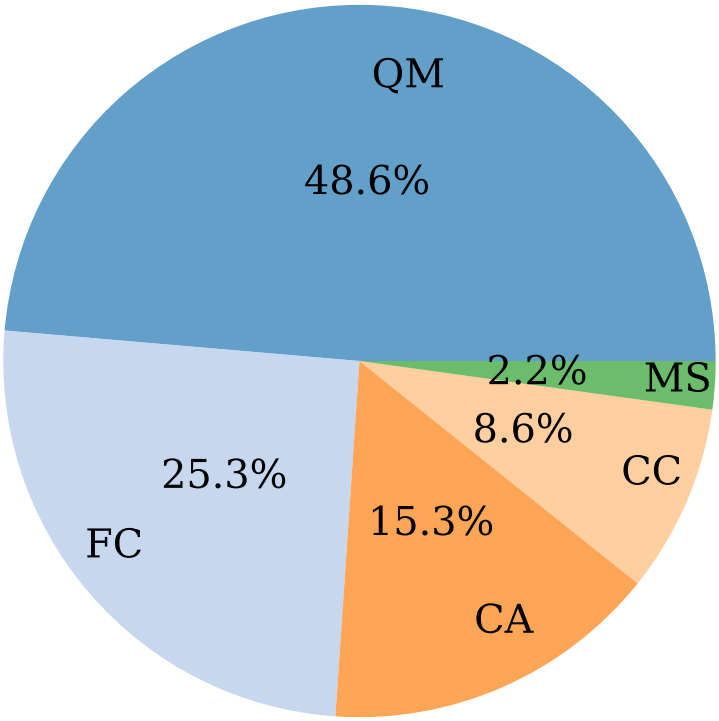


Fig. 3. LLaMA3-8B

13. Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. arXiv preprint arXiv:2410.21276.
14. Weisen Jiang, Han Shi, Longhui Yu, Zhengying Liu, Yu Zhang, Zhenguo Li, and James Kwok. 2024. Forward-backward reasoning in large language models for mathematical verification. In Findings of the Association for Computational Linguistics ACL 2024, pages 6647–6661.
15. Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. Mawps: A math word problem repository. In Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies, pages 1152–1157.
16. Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, et al. 2024. Training language models to self-correct via reinforcement learning. arXiv preprint

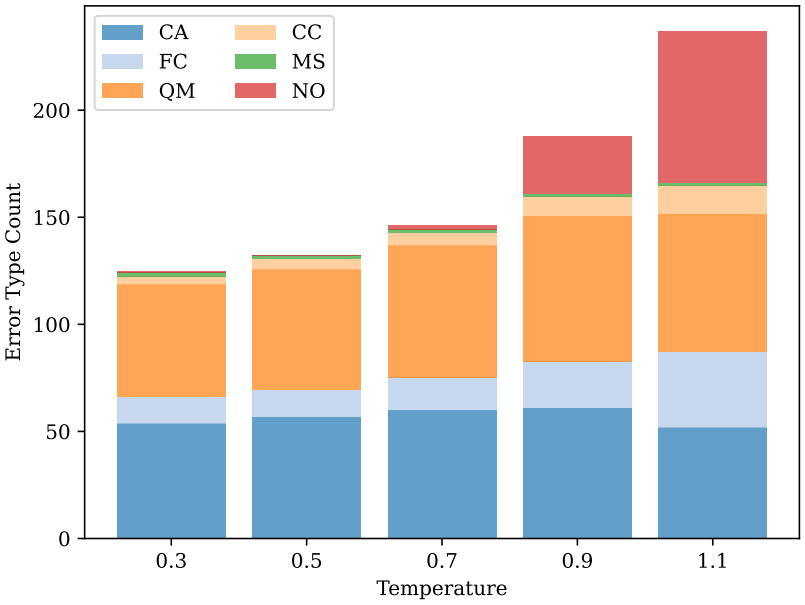


Fig. 4. (a) Error type distribution on MATH.

arXiv:2409.12917.

17. Nicholas Lee, Thanakul Wattanawong, Sehoon Kim, Karttikeya Mangalam, Sheng Shen, Gopala Anumanchipalli, Michael Mahoney, Kurt Keutzer, and Amir Gholami. 2024. Llm2llm: Boosting llms with novel iterative data enhancement. In Findings of the Association for Computational Linguistics ACL 2024, pages 6498–6526.
18. Chen Li, Weiqi Wang, Jingcheng Hu, Yixuan Wei, Nanning Zheng, Han Hu, Zheng Zhang, and Houwen Peng. 2024a. Common 7b language models already possess strong math capabilities. arXiv preprint arXiv:2403.04706.
19. Chengpeng Li, Guanting Dong, Mingfeng Xue, Ru Peng, Xiang Wang, and Dayiheng Liu. 2024b. Dotamath: Decomposition of thought with code assistance and self-correction for mathematical reasoning. CoRR.
20. Chengpeng Li, Zheng Yuan, Hongyi Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. 2024c. Mugglemath: Assessing the impact of query and response augmentation on math reasoning. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 10230–10258.
21. Ming Li, Lichang Chen, Jiuhai Chen, Shwai He, and Tianyi Zhou. 2023. Reflection-tuning: Recycling data for better instruction-tuning. In NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following.
22. Xiaoyuan Li, Wenjie Wang, Moxin Li, Junrong Guo, Yang Zhang, and Fuli Feng. 2024 d. https://doi.org/10.18653/v1/2024.findings-acl.673 Evaluating mathematical reasoning of large language models: A focus on error identification and cor-

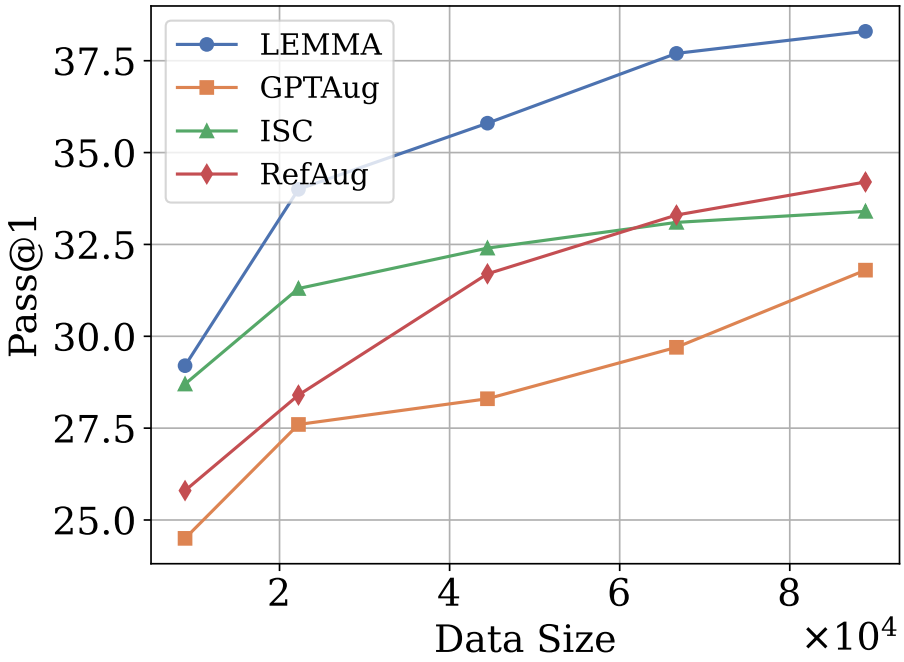


Fig. 5. Pass@1 on MATH.

rection. In Findings of the Association for Computational Linguistics: ACL 2024, pages 11316–11360, Bangkok, Thailand. Association for Computational Linguistics.

23. Yanhong Li, Chenghao Yang, and Allyson Ettinger. 2024e. When hindsight is not 20/20: Testing limits on reflective thinking in large language models. In Findings of the Association for Computational Linguistics: NAACL 2024, pages 3741–3753.
24. Zhenwen Liang, Dian Yu, Wenhao Yu, Wenlin Yao, Zhihan Zhang, Xiangliang Zhang, and Dong Yu. 2024. Mathchat: Benchmarking mathematical reasoning and instruction following in multi-turn interactions. arXiv preprint arXiv:2405.19444.
25. Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In The Twelfth International Conference on Learning Representations.
26. Haoxiong Liu, Yifan Zhang, Yifan Luo, and Andrew C Yao. 2024. Augmenting math word problems via iterative question composing. In ICLR 2024 Workshop on Navigating and Addressing Data Problems for Foundation Models.
27. Zimu Lu, Aojun Zhou, Houxing Ren, Ke Wang, Weikang Shi, Junting Pan, Mingjie Zhan, and Hongsheng Li. 2024 a. https://doi.org/10.18653/v1/2024.acl-long.151 MathGenie: Generating synthetic data with question back-translation for enhancing mathematical reasoning of LLMs. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 2732–2747, Bangkok, Thailand. Association for Computational Linguistics.

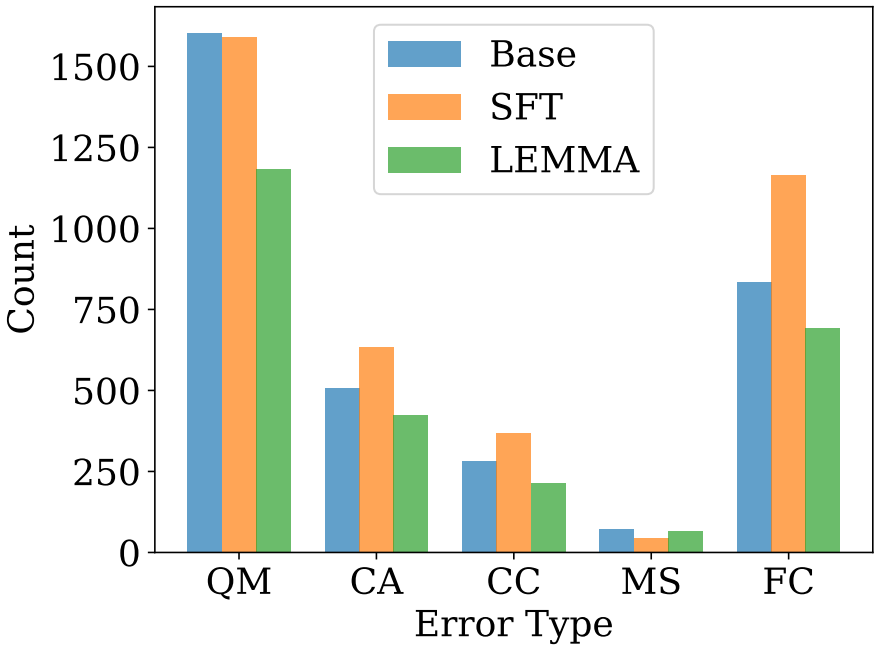


Fig. 6. LLaMA3-MATH

28. Zimu Lu, Aojun Zhou, Ke Wang, Houxing Ren, Weikang Shi, Junting Pan, Mingjie Zhan, and Hongsheng Li. 2024b. Step-controlled dpo: Leveraging stepwise error for enhanced mathematical reasoning. CoRR.
29. Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. arXiv preprint arXiv:2308.09583.
30. Lucie Charlotte Magister, Jonathan Mallinson, Jakub Dominik Adamek, Eric Malmi, and Aliaksei Severyn. 2023. Teaching small language models to reason. In The 61st Annual Meeting Of The Association For Computational Linguistics.
31. Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. https://doi.org/10.18653/v1/2020.acl-main.92 A diverse corpus for evaluating and developing English math word problem solvers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 975–984, Online. Association for Computational Linguistics.
32. Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H Vicky Zhao, Lili Qiu, et al. 2025. On memory construction and retrieval for personalized conversational agents. arXiv preprint arXiv:2502.05589.
33. Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. 2024. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In

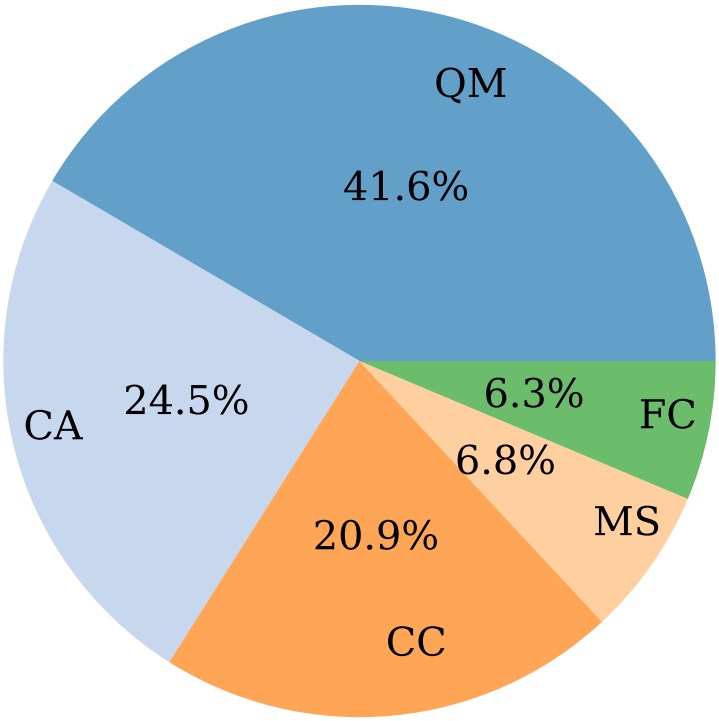


Fig. 7. LLaMA3-8B

Findings of the Association for Computational Linguistics ACL 2024, pages 963–981.

34. Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are nlp models really able to solve simple math word problems? In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 2080–2094.
35. Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. 2024. Mutual reasoning makes smaller llms stronger problem-solvers. arXiv preprint arXiv:2408.06195.
36. Yiwei Qin, Xuefeng Li, Haoyang Zou, Yixiu Liu, Shijie Xia, Zhen Huang, Yixin Ye, Weizhe Yuan, Hector Liu, Yuanzhi Li, et al. 2024. O1 replication journey: A strategic progress report–part 1. arXiv preprint arXiv:2410.18982.
37. Matthew Renze. 2024. https://doi.org/10.18653/v1/2024.findings-emnlp.432 The effect of sampling temperature on problem solving in large language models. In

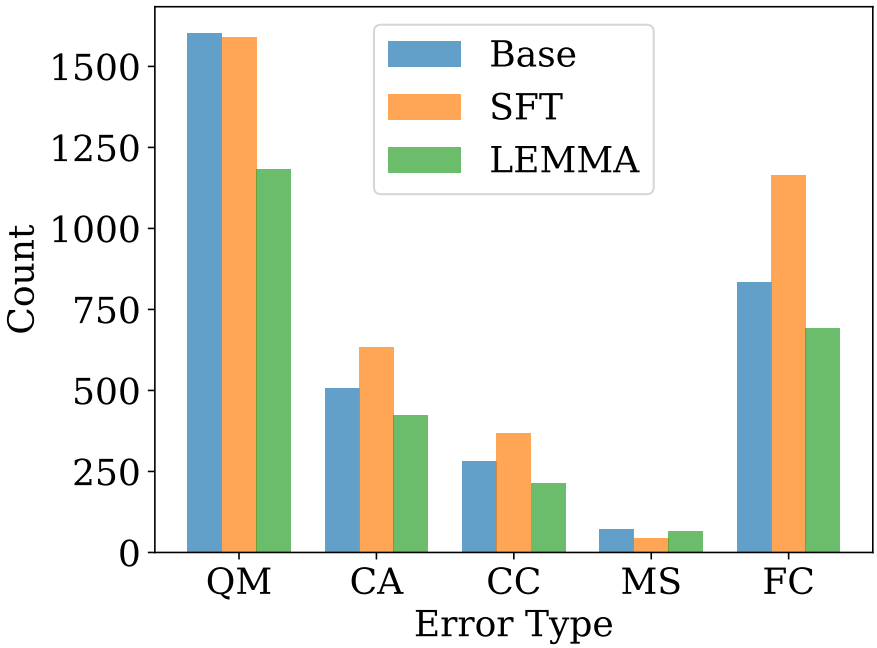


Fig. 8. LLaMA3-MATH

Fig. 9. An example of the two type of solutions. While the initial steps of the brute-force solution are correct, the subsequent process involves intensive computation. Simply correcting the first error step helps a little. Therefore, in the "Fresh & Restart" correction, we encourage the teacher model to rethink from scratch and propose a new solution. All the inter-line equations are manually replaced with inline equations to save space.

Findings of the Association for Computational Linguistics: EMNLP 2024, pages 7346–7356, Miami, Florida, USA. Association for Computational Linguistics.

38. Matthew Renze and Erhan Guven. 2024. Self-reflection in llm agents: Effects on problem-solving performance. arXiv preprint arXiv:2405.06682.
39. Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.
40. Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. 2024. Beyond human data: Scaling self-training for problem-solving with language models. Transactions on Machine Learning Research.
41. Yisheng Song, Ting Wang, Puyu Cai, Subrota K Mondal, and Jyoti Prakash Sahoo. 2023. A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities. ACM Computing Surveys, 55(13s):1–40.

Fig. 10. Example outputs generated by the model (LLaMA3-8B) fine-tuned with LEMMA on GSM8K. LEMMA allows the model to reflect on and correct its previous output or perform careful verification before producing the final answer, as highlighted in bold.

Fig. 11. Example outputs generated by the model (LLaMA3-8B) fine-tuned with LEMMA on MATH. LEMMA allows the model to reflect on and correct its previous output or perform careful verification before producing the final answer, as highlighted in bold.

42. Kaye Stacey, L Burton, and J Mason. 1982. Thinking mathematically. Addison-Wesley London.
43. Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. 2024. Mathscale: Scaling instruction tuning for mathematical reasoning. In Forty-first International Conference on Machine Learning.
44. Yongqi Tong, Dawei Li, Sizhe Wang, Yujia Wang, Fei Teng, and Jingbo Shang. 2024a. Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3065–3080.
45. Yongqi Tong, Sizhe Wang, Dawei Li, Yifan Wang, Simeng Han, Zi Lin, Chengsong Huang, Jiaxin Huang, and Jingbo Shang. 2024b. Optimizing language model's reasoning abilities with weak supervision. arXiv preprint arXiv:2405.04086.
46. Yuxuan Tong, Xiwen Zhang, Rui Wang, Ruidong Wu, and Junxian He. 2024 c. https://openreview.net/forum?id=zLU21oQjD5 DART-math: Difficulty-aware rejection tuning for mathematical problem-solving. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.
47. Huisheng Wang, Zhuoshi Pan, Hangjing Zhang, Mingxiao Liu, Yiqing Lin, and H Vicky Zhao. 2024a. Investalign: Align llms with investor decision-making under herd behavior. In Adaptive Foundation Models: Evolving AI for Personalized and Efficient Learning.
48. Zhilin Wang, Alexander Bukharin, Olivier Delalleau, Daniel Egert, Gerald Shen, Jiaqi Zeng, Oleksii Kuchaiev, and Yi Dong. 2024b. Helpsteer2-preference: Complementing ratings with preferences. arXiv preprint arXiv:2410.01257.
49. Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Shengping Liu, Bin Sun, Kang Liu, and Jun Zhao. Large language models are better reasoners with self-verification. In The 2023 Conference on Empirical Methods in Natural Language Processing.
50. Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Shengping Liu, Bin Sun, Kang Liu, and Jun Zhao. 2023. https://doi.org/10.18653/v1/2023.findings-emnlp.167 Large language models are better reasoners with self-verification. In Findings of the Association for Computational Linguistics: EMNLP 2023, pages 2550–2575, Singapore. Association for Computational Linguistics.
51. Wei Wu, Zhuoshi Pan, Chao Wang, Liyi Chen, Yunchu Bai, Tianfu Wang, Kun Fu, Zheng Wang, and Hui Xiong. 2024a. Tokenselect: Efficient long-context inference and length extrapolation for llms via dynamic token-level kv cache selection. arXiv preprint arXiv:2411.02886.
52. Zhenyu Wu, Qingkai Zeng, Zhihan Zhang, Zhaoxuan Tan, Chao Shen, and Meng Jiang. 2024b. Large language models can self-correct with key condition verifi-

Fig. 12. Representative error type and corresponding examples.

Fig. 13. Error augmentation prompt for introducing flawed reasoning trajectories using the teacher model.

cation. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 12846–12867.

53. Zhiheng Xi, Wenxiang Chen, Boyang Hong, Senjie Jin, Rui Zheng, Wei He, Yiwen Ding, Shichun Liu, Xin Guo, Junzhe Wang, et al. 2024a. Training large language models for reasoning through reverse curriculum reinforcement learning. In Forty-first International Conference on Machine Learning.
54. Zhiheng Xi, Dingwen Yang, Jixuan Huang, Jiafu Tang, Guanyu Li, Yiwen Ding, Wei He, Boyang Hong, Shihan Do, Wenyu Zhan, et al. 2024b. Enhancing llm reasoning via critique models with test-time and training-time supervision. arXiv preprint arXiv:2411.16579.
55. Yifan Xu, Xiao Liu, Xinghan Liu, Zhenyu Hou, Yueyan Li, Xiaohan Zhang, Zihan Wang, Aohan Zeng, Zhengxiao Du, Zhao Wenyi, Jie Tang, and Yuxiao Dong. 2024. https://doi.org/10.18653/v1/2024.findings-emnlp.569 ChatGLM-math: Improving math problem-solving in large language models with a self-critique pipeline. In Findings of the Association for Computational Linguistics: EMNLP 2024, pages 9733–9760, Miami, Florida, USA. Association for Computational Linguistics.
56. Yuchen Yan, Jin Jiang, Yang Liu, Yixin Cao, Xin Xu, Xunliang Cai, Jian Shao, et al. 2024. S3 c-math: Spontaneous step-level self-correction makes large language models better mathematical reasoners. arXiv preprint arXiv:2409.01524.
57. Zhe Yang, Yichang Zhang, Yudong Wang, Ziyao Xu, Junyang Lin, and Zhifang Sui. 2024. Confidence vs critique: A decomposition of self-correction capability for llms. arXiv preprint arXiv:2412.19513.
58. Longhui Yu, Weisen Jiang, Han Shi, YU Jincheng, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024a. Metamath: Bootstrap your own mathematical questions for large language models. In The Twelfth International Conference on Learning Representations.
59. Ping Yu, Jing Xu, Jason E Weston, and Ilia Kulikov. 2024b. Distilling system 2 into system 1. In The First Workshop on System-2 Reasoning at Scale, NeurIPS.
60. Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. 2023. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825.
61. Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2024a. Mammoth: Building math generalist models through hybrid instruction tuning. In The Twelfth International Conference on Learning Representations.
62. Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhu Chen. 2024b. Mammoth2: Scaling instructions from the web. arXiv preprint arXiv:2405.03548.
63. Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022. Star: Bootstrapping reasoning with reasoning. Advances in Neural Information Processing Systems, 35:15476–15488.
64. Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2024 a. https://doi.org/10.18653/v1/2024.findings-acl.924 Small language models need

Fig. 14. "Fix & Continue" correction prompt.

Fig. 15. "Fresh & Restart" correction prompt.

strong verifiers to self-correct reasoning. In Findings of the Association for Computational Linguistics: ACL 2024, pages 15637–15653, Bangkok, Thailand. Association for Computational Linguistics.

65. Zhihan Zhang, Tao Ge, Zhenwen Liang, Wenhao Yu, Dian Yu, Mengzhao Jia, Dong Yu, and Meng Jiang. 2024 b. https://doi.org/10.18653/v1/2024.emnlp-main.817 Learn beyond the answer: Training language models with reflection for mathematical reasoning. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 14720–14738, Miami, Florida, USA. Association for Computational Linguistics.
66. Kun Zhou, Beichen Zhang, Jiapeng Wang, Zhipeng Chen, Wayne Xin Zhao, Jing Sha, Zhichao Sheng, Shijin Wang, and Ji-Rong Wen. 2024. Jiuzhang3. 0: Efficiently improving mathematical reasoning by training small data synthesis models. arXiv preprint arXiv:2405.14365.

Table 1. Error taxonomy modified from ¡cit.¿. Corresponding examples are in Fig.¡ref¿ of appendix ¡ref¿. Some infrequent error types are omitted to save space. For the full taxonomy, please check Tab.¡ref¿ in the appendix.

Error Type	**Definition**
Calculation Error (CA)	Error appears during the calculation process.
Counting Error (CO)	Error occurs during the counting process.
Formula Confusion Error (FC)	Error appears when applying formula in inappropriate scenario.
Question Misinterpretation Error(QM)	Error appears because the **question** is misunderstood, such as ignoring
Missing Step (MS)	Error entails an incomplete generation of reasoning process, lacking a ne
Confusing Concept Error (CC)	Error occurs because two similar but actually different concepts are mist
Nonsensical Output (NO)	Inconceivable, illogical, or question-irrelevant output.

Table 2. Performance comparison on GSM8K, MATH and out-of-distribution datasets. †: numbers reported in ¡cit.¿. The best result is highlighted in bold, and the second best is underlined.

Model	**# Samples**	*In-Distribution*		*Out-Of-Distribution*				
		GSM8K	**MATH**	**ASDIV**	**Mathematics**	**MAWPS**	**SVAMP**	**College-M**
LLaMA3-8B								
SFT	14.97k	65.5	19.3	72.1	23.5	83.0	67.1	13.3
RFT	86.52k	67.3	21.1	74.8	24.9	81.8	69.9	16.7
MetaMath	394.99k	79.2	34.1	81.9	35.3	<u>88.9</u>	76.1	20.5
GPTAug	88.62k	72.1	31.8	81.2	36.5	85.9	79.7	21.2
ISC	86.78k	70.8	33.4	81.1	31.8	82.3	79.7	20.2
S^3C-Math† (w/ MetaMath)	927k	<u>82.9</u>	33.1	-	-	-	81.8	-
RefAug	29.94k	75.9	32.6	82.3	35.5	88.4	81.5	21.0
RefAug-90k	89.92k	77.4	34.2	82.1	35.7	87.7	81.8	21.9
LEMMA	88.90k	79.2	<u>38.3</u>	<u>84.2</u>	<u>39.2</u>	88.8	<u>82.6</u>	**24.7**
LEMMA (w/ MetaMath)	403.59k	**86.4**	**42.3**	**87.1**	**45.8**	**89.5**	**82.8**	<u>24.3</u>
DeepSeekMath-7B								
SFT	14.97k	68.1	35.2	80.9	39.6	88.1	68.1	28.8
RFT	86.52k	73.3	39.3	85.2	46.2	89.3	70.9	31.7
MetaMath	394.99k	79.4	42.0	87.8	49.0	90.2	79.4	31.6
GPTAug	88.62k	77.8	45.5	88.7	52.6	89.6	71.0	31.0
ISC	86.78k	66.3	36.8	82.2	43.1	89.3	71.2	32.0
S^3C-Math† (w/ MetaMath)	927k	<u>82.5</u>	41.4	-	-	-	**82.2**	-
RefAug	29.94k	75.5	39.5	81.2	56.9	82.1	72.8	30.4
RefAug-90k	89.92k	76.7	42.5	82.4	57.5	83.1	74.1	30.6
LEMMA	88.90k	80.4	<u>50.6</u>	<u>89.8</u>	<u>61.6</u>	<u>90.9</u>	81.6	**35.6**
LEMMA (w/ MetaMath)	403.59k	**83.0**	**51.7**	**90.4**	**65.8**	**91.9**	<u>82.1</u>	<u>35.2</u>

Table 3. Evaluation on reflective math reasoning using LLaMA3-8B. "DART-Math-U." denotes DART-MATH-Uniform and "DART-Math-H." denotes DART-MATH-Hard. Best result is highlighted in bold.

Method	**MathChat-FQA**			**MathChat-EC**
	1st	*2nd*	*3rd*	
SFT	63.2	37.2	28.3	66.1
RFT	64.0	40.8	29.1	62.6
MetaMath	80.3	49.4	40.1	66.6
GPTAug	78.8	45.6	37.9	80.0
ISC	78.4	46.8	39.3	78.5
RefAug	69.3	45.0	36.5	82.7
RefAug-90k	71.7	47.0	38.6	83.8
DART-Math-U.	82.4	**50.3**	37.1	80.6
DART-Math-H.	81.7	46.2	35.1	79.8
LEMMA	**83.4**	49.4	**43.4**	**84.7**

Table 4. Performance comparison between different variants of LEMMA and DART-MATH on LLaMA3-8B. "DART-Math-H. (90k)" denotes a $90.7k$ dataset downsampled from DART-MATH-Hard. "LEMMA-Hard" denotes integrating LEMMA with DART-MATH.

Method	Size	GSM8K	MATH	Avg.
DART-Math-H. (90k)	90.7k	81.7	42.6	62.1
LEMMA	88.9k	79.2	38.3	58.7
DART-Math-H. (585k)	585.0k	81.1	**46.6**	63.8
LEMMA-Hard	90.7k	**83.1**	44.8	**63.9**

Table 5. Ablation study on each component of LEMMA.

Method	GSM8K	MATH
LEMMA	**79.2**	**38.3**
w/o Error Aug.	72.8	28.8
w/o Error Aug. (90k)	74.3	32.1
w/o Fresh & Restart	75.2	34.4
w/o Fresh & Restart (90k)	75.7	34.1
w/o Truncation	69.4	22.8

Table 6. Performance comparison with additional baselines on GSM8K, MATH using LLaMA3-8B.

Method	GSM8K	MATH	Avg.
LEMA [2]	75.1	32.8	53.9
Mistake tuning [44]	68.9	25.5	47.2
LLM2LLM [17]	76.8	33.1	54.9
LEMMA	**79.2**	**38.3**	**58.7**

Table 7. Data statistics of the different methods. †: solutions generated by LLaMA3, ⋆: numbers reported in ¡cit.¿.

Method	# Solution	# Avg. Token
SFT	14.97k	242.85
RFT† [60]	86.52k	172.96
MetaMath [58]	394.99k	231.08
GPTAug [1]	88.62k	619.54
ISC [9]	86.78k	746.71
S^3C-Math [56] (w/ MetaMath)	927k⋆	-
RefAug [65]	29.94k	737.94
RefAug++ [65]	89.82k	770.38
LEMMA	88.90k	623.38
LEMMA (w/ MetaMath)	403.59k	544.14

Table 8. Performance comparison on GSM8K, MATH, and out-of-distribution datasets uses LLaMA-3.1-Nemotron-70B as the teacher model. LEMMA demonstrates robustness to the choice of teacher model.

Model	*In-Distribution*		*Out-Of-Distribution*					Avg.
	GSM8K	MATH	ASDIV	Mathematics	MAWPS	SVAMP	College-Math	
LLaMA-3-8B								
GPTAug	72.1	31.8	81.2	36.5	85.9	79.7	21.2	58.3
LEMMA (GPT-4o)	79.2	38.3	84.2	39.2	88.8	82.6	24.7	62.4
LEMMA (LLaMA3)	77.3	36.4	84.1	37.9	87.7	82.7	23.1	61.3
DeepSeekMath-7B								
GPTAug	77.8	45.5	88.7	52.6	89.6	71.0	31.0	65.2
LEMMA (GPT-4o)	80.4	50.6	89.8	61.6	90.9	81.6	35.6	70.1
LEMMA (LLaMA3)	78.4	48.8	88.8	60.7	88.0	76.1	34.7	67.9

Table 9. Additional results on GSM8K, MATH and out-of-distribution datasets using Mistral-7B-v0.1 and Qwen2-Math-7B. †: numbers reported in ¡cit.¿.

Model	# Samples	*In-Distribution*		*Out-Of-Distribution*				
		GSM8K	MATH	ASDIV	Mathematics	MAWPS	SVAMP	College-M
Mistral-7B-v0.1								
SFT	14.97k	56.4	14.1	62.2	16.6	72.6	52.6	9.2
RFT	86.52k	55.6	12.7	65.5	16.6	73.8	57.4	9.5
MetaMath	394.99k	72.6	28.1	75.9	26.6	85.0	69.4	15.4
GPTAug	88.62k	69.0	30.9	77.6	34.6	82.2	71.6	16.7
ISC	86.78k	54.1	24.6	18.1	27.4	19.5	12.2	14.3
RefAug	29.94k	71.9	30.7	78.4	33.7	83.7	74.7	17.7
RefAug-90k	89.92k	73.0	31.4	79.9	34.8	**86.1**	78.1	17.5
LEMMA	88.90k	**80.8**	**34.5**	**81.1**	**40.3**	85.8	**78.9**	**20.1**
Qwen2-Math-7B								
SFT	14.97k	78.7	50.9	88.1	50.3	92.4	78.9	37.0
RFT	86.52k	83.5	54.4	90.7	57.4	92.7	80.0	38.5
MetaMath	394.99k	84.2	51.8	90.4	60.7	92.6	81.9	34.4
GPTAug	88.62k	83.8	53.6	92.3	64.9	**95.2**	**89.5**	36.6
ISC	86.78k	77.1	48.9	89.4	51.9	92.1	78.3	31.6
S^3C-Math† (w/ MetaMath)	927k	84.7	51.7	-	-	-	87.4	-
RefAug	29.94k	80.1	53.5	92.0	62.7	92.9	80.5	35.1
RefAug-90k	89.92k	84.1	56.4	92.4	68.7	93.2	84.3	36.2
LEMMA	88.90k	**87.4**	**62.9**	**93.0**	**74.1**	94.8	88.9	**39.1**

Table 10. Performance comparison on GSM8K, MATH and out-of-distribution datasets under Pass@1 and Majority@32 (Maj@32) settings. †: numbers reported in ¡cit.¿. The best result is highlighted in bold. For Maj@32 evaluation, temperature is 0.7.

Model	# Samples	GSM8K		MATH		ASDIV		Mathematics		Coll
		Pass@1	*Maj@32*	*Pass@1*	*Maj@32*	*Pass@1*	*Maj@32*	*Pass@1*	*Maj@32*	*Pass@*
LLaMA3-8B										
SFT	14.97k	65.5	80.3	19.3	30.8	72.1	82.3	23.5	35.0	13.3
RFT	86.52k	67.3	79.3	21.1	29.4	74.8	84.5	24.9	37.2	16.7
MetaMath	394.99k	79.2	85.7	34.1	42.2	81.9	87.9	35.3	47.3	20.5
GPTAug	88.62k	72.1	81.1	31.8	38.8	81.2	**90.4**	36.5	47.5	21.2
ISC	86.78k	70.8	85.5	33.4	48.4	81.1	87.4	31.8	47.8	20.2
S^3C-Math† (w/ MetaMath)	927k	**82.9**	87.3	33.1	41.6	-	-	-	-	-
RefAug	29.94k	75.9	83.5	32.6	42.7	82.3	90.0	35.5	46.6	21.0
RefAug-90k	89.92k	77.4	85.3	34.2	41.8	82.1	90.3	35.7	49.3	21.9
LEMMA	88.90k	79.2	**90.3**	**38.3**	**53.1**	**84.2**	90.1	**39.2**	**53.3**	**24.7**
DeepSeekMath-7B										
SFT	14.97k	68.1	81.7	35.2	48.6	80.9	90.0	39.6	55.1	28.8
RFT	86.52k	73.3	84.5	39.3	50.7	85.2	91.6	46.2	62.9	31.7
MetaMath	394.99k	79.4	84.8	42.0	52.5	87.8	93.1	49.0	64.2	31.6
GPTAug	88.62k	77.8	87.2	45.5	55.2	88.7	93.7	52.6	75.6	31.0
ISC	86.78k	66.3	82.5	36.8	51.4	82.2	91.4	43.1	61.6	32.0
S^3C-Math† (w/ MetaMath)	927k	**82.5**	88.2	41.4	52.1	-	-	-	-	-
RefAug	29.94k	75.5	86.5	39.5	49.5	81.2	94.1	56.9	70.1	30.4
RefAug-90k	89.92k	76.7	86.7	42.5	50.6	82.4	**94.3**	57.5	73.1	30.6
LEMMA	88.90k	80.4	**89.2**	**50.6**	**63.6**	**89.8**	93.1	**61.6**	**77.4**	**35.6**

Table 11. Ablation study on each component of LEMMA. Removing either the mistake augmentation module or the fresh & restart module results in a dramatic decline in performance.

Method	GSM8K	MATH	ASDIV	Mathematics	MAWPS	SVAMP	College-Math	Avg.
LEMMA	**79.2**	**38.3**	**84.2**	**39.2**	**88.8**	**82.6**	**24.7**	**62.4**
w/o Error Aug.	72.8	28.8	81.0	32.0	85.1	75.6	21.7	56.7
w/o Error Aug. (90k)	74.3	32.1	81.2	35.8	86.5	78.5	21.8	58.6
w/o Fresh & Restart	75.2	34.4	82.7	37.1	87.7	79.6	22.2	59.8
w/o Fresh & Restart (90k)	75.7	34.1	82.3	37.6	87.0	80.2	22.1	59.9
w/o Truncation	69.4	22.8	77.1	29.4	85.1	75.6	18.7	54.0

Table 12. Error taxonomy adapted from ¡cit.¿. We add "Question Misinterpretation Error (QM)", "Confusing Concept Error (CC)", and "Nonsensical Output (NO)" to allow for a more fine-grained identification of error types. Corresponding examples are provided in Figure ¡ref¿.

Error Type	Definition
Calculation Error (CA)	Error appears during the calculation process.
Counting Error (CO)	Error occurs during the counting process.
Context Value Error (CV)	Error arises when attributes of named entities do not align with the info
Hallucination (HA)	Error involves adding fictitious unrelated statements contradictory to th
Unit Conversion Error (UC)	Error occurs during unit conversion process.
Operator Error (OP)	Error involves a single operator being erroneously applied within the exp
Formula Confusion Error (FC)	Error appears when applying formula in inappropriate scenario.
Missing Step (MS)	Error entails an incomplete generation of reasoning process, lacking a ne
Contradictory Step (CS)	Error manifests inconsistency between preceding and subsequent reasoni
Question Misinterpretation Error(QM)	Error appears because the **question** is misunderstood, such as ignoring
Confusing Concept Error (CC)	Error occurs because two similar but actually different concepts are mist
Nonsensical Output (NO)	Inconceivable, illogical, or question-irrelevant output.